

Chapter 1

Tuesday, January 30, 2024 8:32 AM

- Purpose of Parallel Computing
 - Larger problems
 - Faster time to solution
- Simple parallelism
 - No dependency between tasks means we can have more runners
 - EX: Checkout at supermarket
- Serial performance improvements is limited in the future
 - TDP, Clock Freq, Power -> Heat

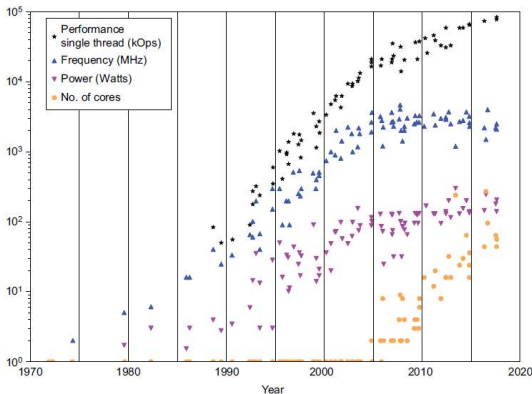


Figure 1.2 Single thread performance, CPU clock frequency (MHz), CPU power consumption (watts), and the number of CPU cores from 1970 to 2018. The parallel computing era begins about 2005, when the core count in CPU chips begins to rise, while the clock frequency and power consumption plateaus, yet performance steadily increases (Horowitz et al. and Rupp, <https://github.com/karlrupp/microprocessor-trend-data>).

- CPUs and GPUs as consumer grade processors
 - CPUs
 - Hyperthreading = Two instruction queues
 - Vector processor = Total bit width / vector bit width gives number of parallel operations
 - SIMD / AVX I think?
 - GPUs will be discussed later
- Benefits
 - Speedup: Reducing time to solution is most useful immediate result
 - More nodes: Parallelizing over multiple nodes means larger problems and more access to IO, memory, etc.
 - Power
 - Rather than using one core at high power, use many cores at low power
 - $P = (N \text{ Processors}) * (R \text{ Watts/Processors}) * (T \text{ hours})$
 - Cost: GPUs result in faster results for some uses, meaning GPU time is worth more than CPU time, even if more expensive
- Laws
 - Amdahl's Law: $\text{Speedup}(N) = 1 / (S + P/N)$
 - Strong scaling
 - Limited by serial portion
 - Alternative if size scales to processors: $\text{Speedup}(N) = N - S * (N - 1)$
 - Weak scaling
 - Use larger problems

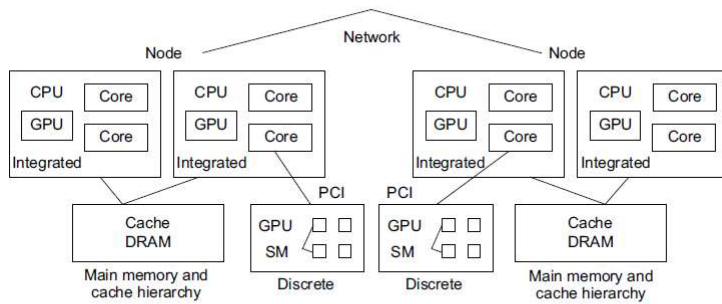


Figure 1.19 A general heterogeneous parallel architecture model consisting of two nodes connected by a network. Each node has a multi-core CPU with an integrated and discrete GPU and some memory (DRAM). Modern compute hardware normally has some arrangement of these components.

- Message Passing
 - MPI
 - User makes processes which communicate in user space
 - OS assigns where processes go (for each core) in kernel space, not directly controlled
 - Hints provided via SLURM or MPI runner (mpirun, srun, mpiexec)
- Threading
 - OpenMP
 - Up to user to decide what parts can be safely parallel
- Vectorization
 - Run blocks of data in parallel
 - Affected strongly by compiler using right instructions
 - Compiler reads pragma or source code hints on what to do
- Stream processing
 - Special purpose processor
 - Reads data in, provides processed data out
 - Offloading to GPU is stream processing

		Instruction	
		Single	Multiple
Data	Single	SISD Single instruction single data	MISD Multiple instruction single data
	Multiple	SIMD Single instruction multiple data	MIMD Multiple instruction multiple data

Figure 1.24 Flynn's Taxonomy categorizes different parallel architectures. A serial architecture is single data, single instruction (SISD). Two categories only have partial parallelization in that either the instructions or data are parallel, but the other is serial.

- Parallel Speedup: Moving from serial to parallel on one type of hardware to compute faster
- Comparative Speedup: Moving from one architecture to another to compute faster
- When discussing performance
 - (GPU 2016: CPU 2016): This indicates GPU an CPU from same year
 - (CPU Best 2016): Indicates high end CPU
 - Still seems improper, listing exact hardware is better. Best AMD Ryzen CPU right now is ambiguous for example